۴-۵ نخ‌سازی ضمنی

با تداوم رشد پردازش چندهسته‌ای، کاربردهایی که صدها–یا حتی هزاران–نخ را در بر می‌گیرند ظهور پیدا می‌کنند. طراحی این قبیل برنامه‌ها کار چندان بدیهی هم نیست: برنامه‌نویسان نه تنها بایستی چالش‌های مطرح شده در بخش ۴-۲ را در نظر بگیرند بلکه مشکلات دیگری نیز هستند. این مشکلات که به درستی برنامه مربوط می‌شوند در فصل ۶ و فصل ۸ پوشش داده می‌شوند.

یک شیوه برای رفع این مشکلات و پشتیبانی بهتر طراحی برنامه‌های همروند و موازی، انتقال ایجاد و مدیریت نخ‌سازی از دوش توسعه‌دهندگان برنامه به کامپایلرها و کتابخانه‌های زمان اجراست. این راهبرد، با اصطلاح **نخ‌سازی ضمنی**، یک نگرش فراگیر روبه‌رشد است. در این بخش، چهار رویکرد جایگزین را برای طراحی برنامه‌هایی که می‌توانند از مزیت پردازنده‌های چندهسته‌ای با نخ‌سازی ضمنی بهره‌مند شوند، بحث و بررسی می‌کنیم. همان‌گونه که خواهیم دید، این راهبردها عموماً لازم دارند که برنامه‌نویسان کاربردی *وظایفی*—نه نخ‌ها—را مشخص کنند که می‌توانند اجرای موازی داشته باشند. یک وظیفه معمولاً به‌صورت یک تابع نوشته می‌شود که در ادامه، کتابخانه‌ی زمان اجرا به یک نخ مجزایی نوعاً با استفاده از مدل چند–به–چند نگاشت می‌دهد (بخش ۴-۳-۳). حسن این رویکرد در آن است که برنامه‌نویسان تنها نیاز دارند که وظایف موازی و کتابخانه‌هایی را شناسایی کنند که جزئیات خاص ایجاد و مدیریت نخ را معین می‌کنند.

۴-۵-۱ انبارهای نخ

در بخش ۴-۱، یک کارگزار وب چندنخی را مطرح کردیم. در اینجا، هر بار که کارگزار درخواستی دریافت می‌کند، نخ مجزایی برای سرویس درخواست ایجاد می‌کند. با این که ایجاد یک نخ مجزا یقیناً از ایجاد یک پردازه‌ی جداگانه بهتر است، اما باز هم یک کارگزار چندنخی مسائل بالقوه‌ای دارد. اولین دغدغه به میزان زمان لازم برای ایجاد نخ، پیش از شروع سرویس درخواست بر می‌گردد، با علم به این حقیقت که با تکمیل کار سرویس بایستی نخ مذکور کنار گذاشته شود. دغدغه‌ی دوم بغرنج‌تر هم هست. اگر اجازه بدهیم تا هر درخواست همروند با یک نخ جدید سرویس داده شود، روی تعداد نخ‌های همروند فعال سیستم محدودیتی نمی‌گذاریم. با داشتن نخ‌های نامحدود، تمامی منابع سیستم از قبیل زمان پردازنده یا حافظه را در عمل از دست خواهیم داد. یک راه‌حل این مسأله استفاده از **انبار نخ**[1] می‌باشد.

انبارهای نخ اندروید

ما در بخش ۳-۸-۲-۱، RPCهای سیستم‌عامل اندروید را پوشش دادیم. شما از آن بخش به یاد دارید که اندروید از زبان تعریف رابط اندروید (AIDL) استفاده می‌کند، ابزاری که رابط دوری را که کارفرماها با آن بر روی کارگزار تعامل برقرار می‌کنند، توصیف می‌کند. AIDL یک انبار نخ هم فراهم می‌کند. یک سرویس دور با استفاده از انبار نخ می‌تواند به درخواست‌های همروند متعددی با سرویس

1. Thread pool

هر درخواست با استفاده از یک نخ مجزایی از انبار رسیدگی کند.

ایده‌ی کلی که پشت یک انبار نخ وجود دارد این است که تعدادی نخ در زمان شروع به کار به وجود آمده و در یک *انبار* قرار داده می‌شوند که در آنجا به انتظار کار می‌نشینند. زمانی‌که کارگزار یک درخواست دریافت می‌کند، به‌جای ایجاد یک نخ، درخواست را در عوض به انبار نخ ارسال کرده و انتظارکشی درخواست‌های بیشتر را از سر می‌گیرد. اگر نخی در انبار در دسترس بود، بیدار می‌گردد، و بلافاصله درخواست سرویس می‌شود. اگر نخی در انبار موجود نباشد، وظیفه منتظر می‌ماند تا این که یکی آزاد شود. همین که یک نخ سرویسش را تکمیل می‌کند، به انبار برگشته و منتظر کار بیشتری می‌ماند. انبارهای نخ خوب عمل می‌کنند زمانی که وظایف ارسالی به انبار بتوانند به‌صورت ناهمگام اجرا شوند.

انبارهای نخ منافع زیر را به دنبال دارند:

۱- سرویس یک درخواست با یک نخ موجود، اغلب سریع‌تر از انتظار ایجاد یک نخ می‌باشد.
۲- انبار نخ، تعداد نخ‌هایی را که در هر لحظه می‌توانند وجود داشته باشند، محدود می‌کند. این مورد به ویژه در سیستم‌هایی که نمی‌توانند تعداد نخ‌های همروند زیادی را پشتیبانی کنند، اهمیت دارد.
۳- تفکیک وظیفه‌ای که بایستی انجام شود از فرایند چگونگی ایجاد وظیفه، به ما امکان می‌دهد که از راهبردهای متفاوتی برای اجرای وظیفه استفاده کنیم. برای مثال، وظیفه می‌تواند زمانبندی شود تا بعد از یک تأخیر زمانی اجرا شده یا اجرای متناوبی داشته باشد.

تعداد نخ‌های انبار می‌تواند با ابتکاری از روی عواملی چون تعداد پردازنده‌های سیستم، میزان حافظه‌ی فیزیکی و تعداد درخواست‌های همروند کارفرماهای مورد انتظار انتخاب شود. در معماری‌های پیچیده‌تر انبار نخ می‌توان تعداد نخ‌های انبار را مطابق الگوهای آماری استفاده به صورت پویا انتخاب کرد. مزیت دیگر این قبیل معماری‌ها آن است که در موقع پایین بودن بار سیستم، انبار کوچک‌تری —که با این کار، مصرف حافظه هم پایین می‌آید—فراهم می‌شود. یک معماری این مدلی در **اعزام مرکزی بزرگ**[1] شرکت اپل وجود دارد که بعداً در همین بخش بحث می‌کنیم.

API ویندوز توابع متعددی را در رابطه با انبارهای نخ فراهم می‌کند. استفاده از API انبار نخ مشابه ایجاد یک نخ با تابع CreateThread() می‌باشد که در بخش ۴-۴-۲ مطرح شد. در اینجا، تابعی که قرار است به صورت نخ مجزایی اجرا داشته باشد، تعریف می‌شود. چنین تابعی می‌تواند ظاهری به شکل زیر داشته باشد:

```
DWORD WINAPI PoolFunction(PVOID Param) {
        /**
        * this function runs as a separate thread.
        **/
}
```

1. Grand Central Dispatch

اشاره‌گری به تابع PoolFunction() برای یکی از توابع API انبار نخ ارسال شده و این تابع توسط نخی از انبار اجرا می‌شود. یکی از اعضای این چنینی API انبار نخ، تابع QueueUserWorkItem() است که سه پارامتر زیر به آن ارسال می‌شوند:

- LPTHREAD_START_ROUTINE Function اشاره‌گری به تابعی که قرار است به عنوان نخ مجزایی اجرا شود.
- PVOID Param پارامتری که به Function ارسال می‌شود.
- ULONG Flags پرچم‌هایی که نشان می‌دهند، انبار نخ چگونه قرار است نخ را ایجاد و اداره کند.

نمونه‌ای از احضار یک تابع به صورت زیر است:

```
QueueUserWorkItem(&PoolFunction, NULL, 0);
```

این فراخوانی باعث می‌شود نخی از انبار نخ، تابع PoolFunction() را از طرف برنامه‌نویس احضار کند. در اینجا، هیچ پارامتری به PoolFunction() ارسال نمی‌کنیم. چون پرچم را ۰ انتخاب کرده‌ایم، دستورالعمل خاصی برای انبار نخ جهت ایجاد نخ هم نداریم.

اعضای دیگر API انبار نخ ویندوز شامل برنامه‌های سودمندی هستند که توابعی را با فواصل متناوب یا در زمان تکمیل ناهمگام یک درخواست I/O احضار می‌کنند.

۴-۵-۱-۱ انبارهای نخ جاوا

بسته‌ی java.util.concurrent شامل یک API برای گونه‌های متعددی از معماری‌های انبار نخ می‌باشد. در اینجا، ما بر سه مدل زیر تمرکز می‌کنیم:

۱- executor نخ منفرد—newSingleThreadExecutor()—انبار نخی به اندازه‌ی ۱ ایجاد می‌کند.
۲- executor نخ ثابت—newFixedThreadPool(int size)—یک انبار نخی با تعداد مشخصی نخ ایجاد می‌کند.
۳- executor نخ کَش‌شده—newCachedThreadPool()—یک انبار نخ بدون محدودیتی، با بازاستفاده‌ی نخ‌ها در نمونه‌های بسیاری ایجاد می‌کند.

ما در حقیقت قبلاً استفاده از یک انبار نخ جاوا را در بخش ۴-۴-۳ دیده‌ایم که در آنجا در برنامه‌ی نمونه‌ی نشان داده شده در شکل ۴-۱۴، یک newSingleThreadExecutor ایجاد کردیم. در آن بخش، متذکر شدیم که چارچوب executor جاوا می‌تواند برای ساخت ابزار نخ‌سازی قوی‌تری استفاده شود. اکنون، چگونگی استفاده از آن را برای ایجاد انبارهای نخ را تشریح می‌کنیم.

یک انبار نخ با استفاده از یکی از متدهای factory کلاس Executors ایجاد می‌شود:

- static ExecutorService newSingleThreadExecutor()
- static ExecutorService newFixedThreadPool(int size)
- static ExecutorService newCachedThreadPool()

هر یک از این متدهای factory، یک نمونه شیئی را ایجاد و بر می‌گردانند که رابط ExecutorService را پیاده‌سازی می‌کند. ExecutorService رابط Executor را گسترش می‌دهد که به ما امکان می‌دهد تا متد execute() را بر روی این شئ احضار کنیم. به‌علاوه، ExecutorService، متدهایی برای مدیریت ختم انبار نخ فراهم می‌کند.

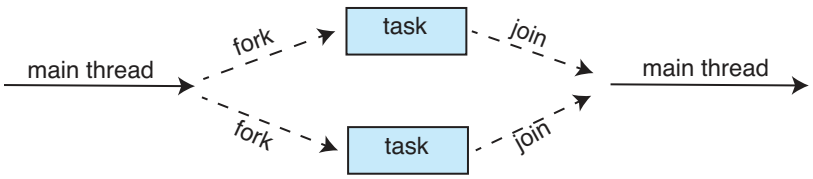
مثال نشان داده شده در شکل ۴-۱۵، یک انبار نخ کَش‌شده ایجاد کرده و با استفاده از متد execute()، وظایفی را به‌منظور اجرا توسط نخی از انبار ارسال می‌کند. موقعی که متد shutdown() احضار می‌شود، انبار نخ، وظایف بیشتر را رد کرده و به‌محض این که تمامی وظایف موجود، اجرایشان را تکمیل کردند انبار بسته می‌شود[1].

```
import java.util.concurrent.*;
public class ThreadPoolExample
{
        public static void main(String[] args) {
        int numTasks = Integer.parseInt(args[0].trim());
        /* Create the thread pool */
        ExecutorService pool = Executors.newCachedThreadPool();
        /* Run each task using a thread in the pool */
        for (int i = 0; i < numTasks; i++)
                pool.execute(new Task());
        /* Shut down the pool once all threads have completed */
        pool.shutdown();
}
```

شکل ۴-۱۵ ایجاد یک انبار نخ در جاوا.

Fork Join ۴-۵-۲

راهبرد ایجاد نخی که در بخش ۴-۴ پوشش داده شد غالباً به مدل **fork-join** معروف است. به یاد بیاورید که با این شیوه، نخ والد اصلی یک یا چند فرزند ایجاد کرده (forks) و در ادامه، به‌انتظار فرزندان می‌ماند تا خاتمه یابند و به آن ملحق شوند، در آنجاست که می‌تواند نتایج آن‌ها را بازیابی و ترکیب کند. این مدل همگام، غالباً تحت عنوان ایجاد نخ صریح شناخته می‌شود، اما یک نامزد ممتازی برای نخ‌سازی ضمنی است. در شرایط دومی، نخ‌ها به‌صورت مستقیم در طی گام fork ساخته نمی‌شوند؛ بلکه، وظایف موازی اعلام می‌شوند. این مدل در شکل ۴-۱۶ نمایش داده می‌شود. یک کتابخانه، تعداد نخ‌هایی را مدیریت می‌کند که ایجاد می‌شوند و مسئولیت گمارش وظایف به نخ‌ها را نیز عهده‌دار است. به یک شیوه‌ای، این مدل fork-join یک نسخه‌ی همگامی از انبارهای نخ است که در آن، یک کتابخانه تعداد واقعی نخ‌هایی را که قرار است ایجاد شوند معین می‌کند—برای مثال، با استفاده از ابتکاری[2] که در بخش ۴-۵-۱ بیان شد.



شکل ۴-۱۶ موازی‌سازی fork-join.

1. Shuts down
2. Heuristics

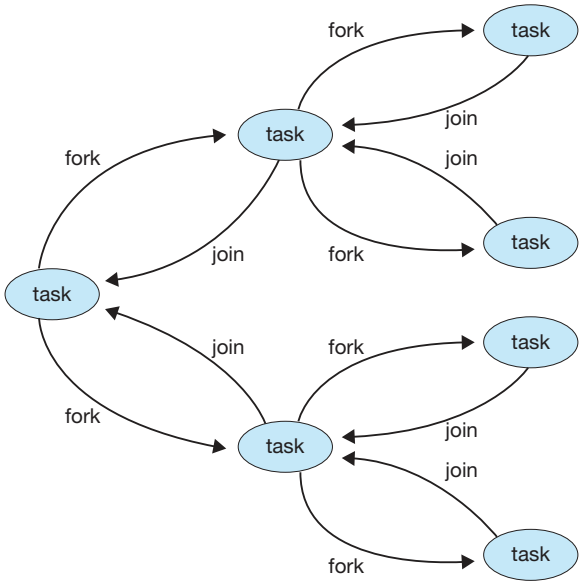
۴-۵-۲-۱ Fork Join در جاوا

جاوا یک کتابخانه‌ی fork-join در نسخه‌ی ۱/۷ از API خود معرفی کرد که طراحی شد تا با الگوریتم‌های تقسیم‌وغلبه‌ی[1] بازگشتی همچون Quicksort و Mergesort استفاده شود. موقع پیاده‌سازی الگوریتم‌های تقسیم‌وغلبه با استفاده از این کتابخانه، وظایف مجزایی در طی گام تقسیم، منشعب شده[2] و به زیرمجموعه‌های کوچک‌تری از مسئله‌ی اصلی گمارده می‌شوند. الگوریتم‌ها بایستی طوری طراحی شوند تا وظایف مجزا بتوانند اجرای همروندی داشته باشند. در یک نقطه‌ای، اندازه‌ی مسئله‌ی گمارده شده به یک وظیفه آن‌قدر کوچک است که می‌تواند مستقیماً حل شود و دیگر نیازی به وظایف بیشتری نیست. الگوریتم بازگشتی کلی پشت سر مدل fork-join جاوا در زیر نشان داده می‌شود:

```
Task(problem)
        if problem is small enough
                solve the problem directly
        else
                subtask1 = fork(new Task(subset of problem)
                subtask2 = fork(new Task(subset of problem)
                result1 = join(subtask1)
                result2 = join(subtask2)

                return combined results
```

شکل ۴-۱۷ این مدل را به‌تصویر می‌کشد.



شکل ۴-۱۷ fork-join در جاوا.

1. Divide-and-conquer
2. Forked

اکنون، راهبرد fork-join جاوا را با طراحی یک الگوریتم تقسیم‌وغلبه که تمامی عناصر یک آرایه از اعداد صحیح را جمع می‌کند نشان می‌دهیم. در API نسخه‌ی ۱/۷، جاوا یک انبار نخ جدیدی—ForkJoinPool—را معرفی کرد که می‌توان وظایفی را گمارد که از کلاس انتزاعی پایه‌ی ForkJoinTask (که در حال حاضر، ما فرض خواهیم کرد که کلاس SumTask می‌باشد) ارث‌بری داشته باشند. کد زیر یک شئ ForkJoinPool ایجاد کرده و وظیفه‌ی آغازین را از طریق متد ()invoke ارسال می‌کند:

```
ForkJoinPool pool = new ForkJoinPool();
// array contains the integers to be summed
int[] array = new int[SIZE];

SumTask task = new SumTask(0, SIZE - 1, array);
int sum = pool.invoke(task);
```

فراخوانی اولیه به ()invoke در زمان تکمیل، حاصل‌جمع آرایه را بر می‌گرداند.

کلاس SumTask—نشان داده شده در شکل ۴–۱۸—یک الگوریتم تقسیم‌وغلبه پیاده‌سازی می‌کند که محتوای آرایه را با استفاده از fork-join جمع می‌زند. وظایف جدید با استفاده از متد ()fork ایجاد می‌شود، و متد ()compute، محاسباتی را که توسط هر وظیفه انجام می‌شود توصیف می‌کند. متد ()compute احضار می‌شود تا این که بتواند مستقیماً حاصل‌جمع زیرمجموعه‌ای را که گمارده شده محاسبه کند. فراخوانی به ()join مسدود می‌ماند تا این که وظیفه کامل شود، آن موقع است که ()join نتایج محاسبه شده در ()compute را بر می‌گرداند.

توجه کنید که SumTask شکل ۴–۱۸، RecursiveTask را گسترش می‌دهد. راهبرد fork-join جاوا حول کلاس انتزاعی پایه‌ی ForkJoinTask سازمان‌دهی شده و کلاس‌های RecursiveTask و RecursiveAction این کلاس را گسترش می‌دهند. تفاوت اساسی میان این دو کلاس در این است که RecursiveTask یک نتیجه بر می‌گرداند (از طریق مقدار برگشتی مشخص شده در ()compute)، و RecursiveAction نتیجه‌ای بر نمی‌گرداند. رابطه‌ی میان سه کلاس با نمودار کلاس UML در شکل ۴–۱۹ نشان داده می‌شود.

یک موضوع مهمی که بایستی مد نظر قرار داد تعیین زمانی است که مسئله «به حد کافی کوچک است» تا به‌صورت مستقیم حل شود و دیگر نیازی به ایجاد وظایف بیشتر نیست. این امر در SumTask زمانی اتفاق می‌افتد که تعداد عناصر جمع زده شده کمتر از THRESHOLD می‌باشد که در شکل ۴–۱۸، مقدار دلخواه ۱۰۰۰ را انتخاب کرده‌ایم. در عمل، تعیین زمانی که یک مسئله می‌تواند به‌صورت مستقیم حل شود مستلزم تعیین زمان دقیق می‌باشد، چرا که مقدار آن می‌تواند بر اساس پیاده‌سازی فرق کند.

چیزی که در مدل fork-join جاوا جالب است، مدیریت وظایف در کتابخانه است که انباری از نخ‌های کارگر ساخته و بار وظایف را مابین کارگران در دسترس موازنه می‌سازد. در برخی شرایط، هزاران وظیفه وجود دارد، در حالی که نخ‌هایی که کار انجام می‌دهند به تعداد انگشتان دست هم نمی‌رسد (برای مثال، یک نخ مجزا برای هر CPU). به‌علاوه، هر نخ در یک ForkJoinPool،

صفی از وظایف را نگه می‌دارد که منشعب ساخته است و اگر صف یک نخ خالی شود، می‌تواند با استفاده از الگوریتم *دزدی کار*، وظیفه‌ای را از صف نخ دیگری بدزدد، لذا بار کاری وظایف در میان تمامی نخ‌ها موازنه می‌شود.

```
import java.util.concurrent.*;
public class SumTask extends RecursiveTask<Integer>
{
        static final int THRESHOLD = 1000;
        private int begin;
        private int end;
        private int[] array;
        public SumTask(int begin, int end, int[] array) {
                this.begin = begin;
                this.end = end;
                this.array = array;
        }

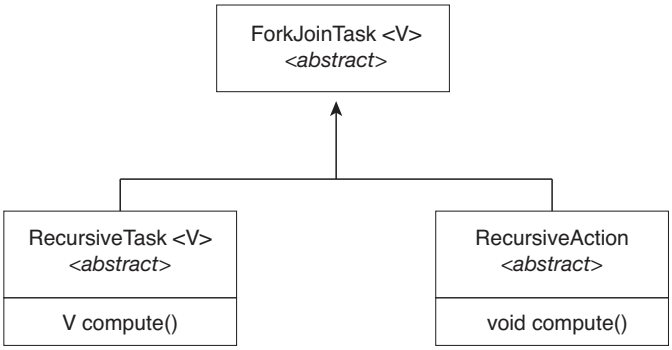
        protected Integer compute() {
        if (end - begin < THRESHOLD) {
                int sum = 0;
                for (int i = begin; i <= end; i++)
                        sum += array[i];
                return sum;
        }
        else {
                int mid = (begin + end) / 2;
                SumTask leftTask = new SumTask(begin, mid, array);
                SumTask rightTask = new SumTask(mid + 1, end, array);
                leftTask.fork();
                rightTask.fork();
                return rightTask.join() + leftTask.join();
        }
        }
}
```

شکل ۴-۱۸ محاسبه‌ی fork-join با استفاده از API جاوا.

OpenMP ۴-۵-۳

OpenMP یک مجموعه دایرکتیو کامپایلر، در کنار یک API برای برنامه‌های نوشته شده به زبان C، C++ یا FORTRAN است که پشتیبانی برنامه‌نویسی موازی را در محیط‌های حافظه‌ی مشترک فراهم می‌کند. OpenMP **نواحی موازی** را به صورت بلاک‌هایی از کد که ممکن است اجرای موازی داشته باشند، شناسایی می‌کند. برنامه‌نویسان کاربردی، دایرکتیوهای کامپایلر را

در کدشان در نواحی موازی وارد می‌کنند و این دایرکتیوها کتابخانه‌ی زمان اجرای OpenMP را هدایت می‌کند تا ناحیه را به صورت موازی اجرا کند. برنامه‌ی C زیر یک دایرکتیو کامپایلر را بالای ناحیه‌ی موازی که شامل دستور printf() است به تصویر می‌کشد:



شکل ۴-۱۹ نمودار کلاس UML برای fork-join جاوا.

```
#include <omp.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
/* sequential code */

        #pragma omp parallel
        {
                printf("I am a parallel region.");
        }

        /* sequential code */

        return 0;
}
```

موقعی که کامپایلر به دایرکتیو

```
        #pragma omp parallel
```

برخورد می‌کند، به تعدادی که هسته‌ی پردازشی در سیستم وجود دارد نخ ایجاد می‌کند. بنابراین، در یک سیستم دوهسته‌ای، دو نخ ایجاد می‌شوند؛ در یک سیستم چهارهسته‌ای، چهار نخ ایجاد می‌شوند؛ و الی آخر. در ادامه، تمامی نخ‌ها به صورت همزمان، ناحیه‌ی موازی را اجرا می‌کنند. هر نخ با خروج از ناحیه‌ی موازی خاتمه می‌پذیرد.

OpenMP چندین دایرکتیو دیگر برای اجرای موازی نواحی کد، از جمله حلقه‌های موازی‌کننده فراهم می‌کند. برای مثال، فرض کنید که دو آرایه‌ی a و b با اندازه‌ی N داریم. تمایل داریم محتوای

آنها را جمع زده و نتایج را در آرایه‌ی c بگذاریم. این کار را می‌توانیم به صورت موازی با استفاده از قطعه کد زیر انجام دهیم که شامل دایرکتیو کامپایلر موازی‌کننده‌ی حلقه‌های for است:

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
}
```

OpenMP کار مشمول حلقه‌ی for را مابین نخ‌هایی که در جواب دایرکتیو

```
#pragma omp parallel for
```

ایجاد کرده است، تقسیم می‌کند.

علاوه بر فراهم‌ساختن دایرکتیوهایی برای موازی‌کردن، OpenMP به برنامه‌نویسان امکان انتخاب را از میان چندین سطح موازی‌سازی فراهم می‌کند. برای مثال، می‌توانند تعداد نخ‌ها را به صورت دستی انتخاب کنند. همین‌طور، به برنامه‌نویسان اجازه می‌دهد تا مشخص کنند که آیا داده‌ها در میان نخ‌ها مشترک هستند یا برای یک نخ خصوصی است. OpenMP در کامپایلرهای متن‌باز و تجاری متعدد برای سیستم‌های لینوکس، ویندوز و macOS در دسترس است.

۴-۵-۴ اعزام مرکزی بزرگ

اعزام مرکزی بزرگ (GCD)، یک فناوری است که توسط اَپل برای سیستم‌عامل‌های macOS و iOS خودش توسعه داده است. آن ترکیبی از یک کتابخانه‌ی زمان اجرا، یک API، و گسترش‌های زبان است که برای برنامه‌نویسان کاربردی، امکان مشخص‌کردن بخش‌هایی از کد (*وظایف*) را که قرار است اجرای موازی داشته باشد فراهم می‌کند. مثل OpenMP، GCD عمده‌ی جزئیات نخ‌سازی را مدیریت می‌کند.

GCD، وظایف را با گذاشتن آن‌ها در یک **صف اعزام**، برای اجرای زمان اجرا زمانبندی می‌کند. موقعی که وظیفه‌ای را از صفی بر می‌دارد، وظیفه را به یک نخ در دسترس از انبار نخ‌هایی که مدیریت می‌کند، می‌گمارد. GCD دو نوع صف اعزام را شناسایی می‌کند: ***سری*** و ***همروند***.

وظایفی که در صف سری گذاشته می‌شوند، به ترتیب FIFO برداشته می‌شوند. همین که یک وظیفه از صف برداشته می‌شود، بایستی قبل از آنکه وظیفه‌ی دیگری برداشته شود، اجرایش را تکمیل کند. هر پردازه صف سری خاص خود را دارد (معروف به **صف اصلی**)، و توسعه‌دهندگان می‌توانند صف‌های سری دیگری ایجاد کنند که برای یک پردازه‌ی خاصی محلی هستند. (به همین دلیل هم هست که چرا صف‌های سری به**صف‌های اعزام خصوصی**[1] معروف هستند.) صف‌های سری برای تضمین اجرای ترتیبی چندین وظیفه مفید هستند.

وظایفی که در یک صف همروند گذاشته می‌شوند نیز به ترتیب FIFO برداشته می‌شوند، اما چندین وظیفه را می‌توان در هر لحظه برداشت، از این رو امکان اجرای موازی وظایف متعدد

1 Private dispatch queues

فراهم می‌شود. چندین صف همروند در سطح سیستم وجود دارند (که به **صف‌های اعزام عمومی**[1] نیز معروف هستند) که به چهار رده‌ی کیفیت سرویس اصلی تقسیم می‌شوند:

- QOS_CLASS_USER_INTERFACE—رده‌ی **user-interactive** معرف وظایفی است که با کاربر تعامل دارند، همچون رابط کاربر و رسیدگی به رویداد، برای تضمین یک رابط کاربر پاسخگو. تکمیل یک وظیفه که به این رده تعلق دارد بایستی فقط یک مقدار کار کوچکی لازم داشته باشد.
- QOS_CLASSS_USER_INITIATED—رده‌ی **user-initiated** شبیه به رده‌ی user-interactive است که در آن، وظایف با یک رابط کاربر پاسخگو مرتبط هستند؛ با این وجود، وظایف user-initiated ممکن است به زمان‌های پردازش طولانی‌تری نیاز داشته باشند. برای مثال، بازکردن یک فایل یا یک URL یک وظیفه‌ی user-initiative است. وظایفی که به این رده تعلق دارند بایستی برای کاربر تکمیل شوند تا تعامل با سیستم را ادامه دهد، اما لزومی ندارد که با همان سرعت وظایف صف user-interactive سرویس شوند.
- QOS_CLASS_UTILITY—رده‌ی **utility** معرف وظایفی است که زمان طولانی‌تری برای تکمیل لازم دارند اما خواهان نتایج آنی نیستند. این رده شامل کارهایی همچون import کردن داده‌هاست.
- QOS_CLASS_BACKGROUND—وظایفی که به رده‌ی **background** تعلق دارند برای کاربر قابل رؤیت نبوده و حساسیت زمانی هم ندارند. مثال‌های این رده شامل ایندکس‌کردن یک سیستم mailbox و انجام کارهای پشتیبان‌گیری است.

وظایفی که به صف‌های اعزام ارسال می‌شوند ممکن است با یکی از دو شیوه‌ی متفاوت زیر بیان شوند:

۱- در زبان‌های C، C++، و Objective-C، GCD یک گسترش زبان، تحت عنوان **بلاک** مشخص می‌کند که به‌سادگی یک واحد خودشمول از کار است. بلاک با یک علامت ^ که در جلوی یک جفت‌آکولاد {} درج می‌شود توصیف می‌شود. کد میان آکولادها واحد کاری را که قرار است انجام شود مشخص می‌کند. مثال ساده‌ای از یک بلاک به‌صورت زیر نشان داده می‌شود:

```
^{ printf("I am a block"); }
```

۲- در زبان برنامه‌نویسی Swift، یک وظیفه با استفاده از یک **بستار**[2] تعریف می‌شود که از این نظر شبیه به یک بلاک است که یک واحد خودشمول از کارکرد را بیان می‌کند. به‌صورت نحوی، یک بستار Swift همانند یک بلاک نوشته می‌شود، منهای علامت ^ جلوی آن.

1. Global dispatch queues
2. Closure

قطعه کد Swift زیر، دست‌یابی به یک صف همروند را برای رده‌ی user-initiated و ارسال یک وظیفه به صف، با استفاده از تابع dispatch_async() نشان می‌دهد:

```
let queue = dispatch_get_global_queue(QOS_CLASS_USER_INITIATED, 0)

dispatch_async(queue,{ print("I am a closure.")})
```

از نظر داخلی، انبار نخ GCD مرکب از نخ‌های پوزیکس می‌باشد. GCD به صورت فعال، انبار را مدیریت کرده و امکان انبساط و انقباض تعداد نخ‌ها را بر اساس تقاضای برنامه‌ها و ظرفیت سیستم فراهم می‌سازد. GCD توسط کتابخانه‌ی libdispatch پیاده‌سازی می‌شود که اَپل تحت لیسانس Apache Commons توزیع کرده است. از همان موقع در سیستم‌عامل FreeBSD پورت شده است.

۴-۵-۵ بلاک‌های سازنده‌ی نخ اینتل

بلاک‌های سازنده‌ی نخ‌سازی[1] (TBB) اینتل یک کتابخانه‌ی template می‌باشد که از طراحی برنامه‌های موازی به‌زبان C++ پشتیبانی می‌کند. چون کتابخانه است به‌هیچ کامپایلر خاص یا پشتیبانی زبان نیازی ندارد. توسعه‌دهندگان وظایفی را که می‌توانند به‌صورت موازی اجرا شوند مشخص می‌کنند و زمان‌بند وظیفه‌ی TBB این وظایف را به نخ‌های ذیربط نگاشت می‌دهد. به‌علاوه، زمان‌بند وظیفه تعادل بار فراهم کرده و آگاه به کَش می‌باشد، به این معنی که تقدم را به وظایفی خواهد داد که احتمالاً داده‌های خود را در حافظه‌ی کَش ذخیره دارند و لذا اجرای سریع‌تری خواهند داشت. TBB مجموعه‌ای غنی از امکانات فراهم می‌کند اعم از templateهایی برای ساختارهای موازی حلقه، عملیات اَتمیک، و قفل‌گذاری انحصار متقابل[2]. به‌علاوه، ساختمان‌داده‌های همروندی فراهم می‌کند، اعم از یک نقشه‌ی درهم، صف، و بردار، که می‌توانند نسخه‌های امن نخی هم‌ارز ساختمان‌داده‌های کتابخانه‌ی template استاندارد C++ را سرویس بدهند.

اجازه بدهید تا موازی‌سازی را برای حلقه‌ها به‌عنوان یک مثال استفاده کنیم. در ابتدا، فرض کنید که تابعی به‌نام apply(float value) وجود دارد که عملیاتی بر روی پارامتر value انجام می‌دهد. اگر ما یک آرایه‌ی v با اندازه‌ی n شامل مقادیر float داشته باشیم، می‌توانستیم از سری زیر برای ایجاد حلقه به‌منظور ارسال هر یک از مقادیر v به تابع apply() استفاده کنیم:

```
for (int i = 0; i < n; i++) {
        apply(v[i]);
}
```

1. Threading building blocks (TBB)
2. Mutual exclusion locking

یک برنامه‌نویس می‌توانست با گماردن نواحی متفاوت آرایه‌ی v به هر کدام از هسته‌های پردازشی، موازی‌سازی داده‌ها (بخش ۴-۲-۲) را به‌صورت دستی روی یک سیستم چندهسته‌ای اِعمال کند؛ با این وجود، این کار، تکنیک دست‌یابی به موازی‌سازی را به‌صورت تنگاتنگ به سخت‌افزار فیزیکی گره می‌زند و لازم است که الگوریتم اصلاح شود و به‌تعداد هسته‌های پردازشی بر روی هر معماری خاصی دومرتبه کامپایل شود.

به‌روشی دیگر، یک برنامه‌نویس می‌توانست از TBB استفاده کند که الگوی parallel_for را فراهم می‌کند که دو مقدار انتظار دارد:

```
parallel_for (range body )
```

که *range* به بازه‌ی عناصری ارجاع می‌کند که تکرار خواهند شد (تحت عنوان **فضای تکرار**[1]) و *body* عملیاتی را مشخص می‌کند که بر روی زیربازه‌ای از عناصر انجام خواهد شد.

اکنون می‌توانیم سری بالا برای حلقه را با استفاده از الگوی parallel_for به‌صورت زیر بنویسیم:

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```

دو پارامتر اول مشخص می‌کنند که فضای تکرار از 0 تا n – 1 می‌باشد (که متناظر تعداد عناصر آرایه‌ی v می‌باشد). پارامتر دوم، یک تابع lambda به‌زبان C++ است که کمی توضیح لازم دارد. عبارت [=] (size_t i) پارامتر i می‌باشد که هر یک از مقادیر فضای تکرار را فرض دارد (در اینجا از 0 تا n - 1). هر مقدار i استفاده می‌شود تا مشخص کند که کدام عنصر v به‌عنوان یک پارامتر به تابع apply(v[i]) ارسال می‌شود.

کتابخانه‌ی TBB تکرارهای حلقه را به «قالب‌هایی» تقسیم خواهد کرد و تعدادی وظیفه ایجاد می‌کند که روی آن قالب‌ها عمل می‌کنند. (تابع parallel_for به برنامه‌نویسان اجازه می‌دهد تا به‌صورت دستی اندازه‌ی قالب‌ها را مشخص کنند در صورتی که تمایل داشته باشند.) TBB تعدادی نخ نیز ایجاد کرده و وظایف را به نخ‌های در دسترس خواهد گماشت. این موضوع کاملاً مشابه کتابخانه‌ی fork-join در جاوا می‌باشد. حسن این رویکرد در این است که تنها لازم دارد برنامه‌نویسان مشخص کنند چه عملیاتی می‌توانند اجرای موازی داشته باشند (با توصیف حلقه‌ی parallel_for) و کتابخانه است که جزئیات مشمول تقسیم کار به وظایف جداگانه را که اجرای موازی دارند مدیریت می‌کند. TBB اینتل داری هر دو نسخه‌ی تجاری و متن‌باز می‌باشد که بر روی ویندوز، لینوکس، و macOS اجرا می‌شوند.

۴-۶ ملاحظات نخ‌سازی

در این بخش، برخی ملاحظات مربوط به طراحی برنامه‌های چندنخی را بحث می‌کنیم.

۴-۶-۱ فراخوانی‌های سیستمی fork() و exec()

در فصل ۳، چگونگی استفاده از فراخوانی سیستمی ()fork را برای ایجاد کپی مجزایی از یک

1. Iteration space